

Design Patterns

Design patterns are typical solutions to common problems in software design. Each pattern is like a blueprint that you can customize to solve a particular design problem in your code.

Creational Patterns

1. Singleton

The Singleton pattern solves two problems at the same time, violating the Single Responsibility Principle.

- Ensure that a class has just a single instance
- Provide a global access point to that instance.

All implementations of the Singleton have these two steps in common:

- Make the **default constructor private**, to prevent other objects from using the new operator with the Singleton class.
- Create a **static creation method that acts as a constructor**. Under the hood, this method calls the private constructor to create an object and saves it in a **static field**. All following calls to this method return the cached object.

Use Case: Use the Singleton pattern when a class in your program should have **just a single instance** available to all clients; for example, a single database object shared by different parts of the program.

Use the Singleton pattern when you need **stricter control over global variables**.

Pros:

- Being sure to get a single instance of an object
- Getting a global access point to that instance
- Creating the Singleton object only when the method is called for the first time.

Cons:

- Violates Single Responsibility Principle
- Singleton can mask bad design in the code such as when the components in a program know too much about each other.
- The client code becomes hard to unit test
- Needs special treatment in multithreaded environment

2. Factory

Factory Method is a creational design pattern that provides an interface for creating objects in a superclass, but allows subclasses to alter the type of objects that will be created.

It has 4 components-

- Creator Class (Can be abstract)
- Concrete Creator class
- Product (Interface)
- Concrete Product

Use Case:

- Use the Factory Method when you don't know beforehand the exact types and dependencies of the objects your code should work with.
- Use the Factory Method when you want to provide users of your library or framework with a way to extend its internal components.
- Use the Factory Method when you want to save system resources by reusing existing objects instead of rebuilding them each time.

Pros:

- Avoids tight coupling between creator and concrete products
- Single Responsibility Principle
- Open Closed Principle

Cons:

- Complicated code because of all the new subclasses

3. Abstract Factory

Abstract Factory is a creational design pattern that lets you produce families of related objects without specifying their concrete classes.

Components:

- Abstract Factory
- Abstract Product
- Concrete Factory
- Concrete Product

Use Case: various families of related products

Pros:

- Product compatibility + same as factory

Cons:

- Code becomes complicated due to a lot of new classes and interfaces.

4. Prototype

Prototype is a creational design pattern that lets you copy existing objects without making your code dependent on their classes. The Prototype pattern **delegates the cloning process to the actual objects that are being cloned**. The pattern declares a common interface for all objects that support cloning. This interface lets you clone an object without coupling your code to the class of that object.

Use Case:

- Use the Prototype pattern when your code shouldn't depend on the concrete classes of objects that you need to copy.
- Use the pattern when you want to reduce **the number of subclasses that only differ in the way they initialize** their respective objects.

Pros:

- Clone objects without coupling to concrete classes
- Getting rid of repeated initialization code
- Producing complex objects more conveniently
- Alternative to inheritance when dealing with configuration presets

Cons:

- Cloning complex objects with circular reference might be very tricky

5. Builder

Builder is a creational design pattern that lets you construct complex objects step by step. The pattern allows you to produce different types and representations of an object using the same construction code. Use for **complex object that requires laborious, step-by-step initialization of many fields and nested objects**.

The pattern organizes object construction into a set of steps

Use Case:

- To get rid of bloated constructors with several optional parameters or several overloaded constructors
- to be able to create different representations of some product
- Use the Builder to construct Composite trees or other complex objects.

Pros:

- SRP
- Construct objects step-by-step, defer construction steps or run steps recursively.
- Reuse the same construction code to build various representations of products.

Cons:

- Increased complexity

Structural Patterns

1. Adapter

Adapter is a structural design pattern that allows **objects with incompatible interfaces** to collaborate. Adapter is a special object that converts the interface of one object so that another object can understand it.

Object Adapter: This implementation uses the object composition principle: the adapter implements the interface of one object and wraps the other one. It can be implemented in all popular programming languages.

Class Adapter: This implementation uses inheritance: the adapter inherits interfaces from both objects at the same time. Note that this approach can only be implemented in programming languages that support multiple inheritance, such as C++.

Use Case:

- Incompatible interface with existing code
- Use the pattern when you want to reuse several existing subclasses that lack some common functionality that can't be added to the superclass.

Pros:

- SRP + OCP

Cons:

- Increased Complexity

With Adapter you access an existing object via different interface. With Proxy, the interface stays the same. With Decorator you access the object via an enhanced interface. Facade defines a new interface for existing objects.

2. Composite

Composite is a structural design pattern that lets you compose objects into tree structures and then work with these structures as if they were individual objects.

Use Case:

- Tree Like Object Structure
- When you want the client code to treat both simple and complex elements uniformly.

Pros:

- OCP
- Working with complex tree structures more conveniently: use polymorphism and recursion to your advantage.

Cons:

- It might be difficult to provide a common interface for classes whose functionality differs too much.

A Decorator is like a Composite but only has one child component. There's another significant difference: Decorator adds additional responsibilities to the wrapped object, while Composite just "sums up" its children's results.

3. Decorator

Decorator is a structural design pattern that lets you attach new behaviors to objects by placing these objects inside special wrapper objects that contain the behaviors. The wrapper implements the same interface as the wrapped object. That's why from the client's perspective these objects are identical.

Use Case:

- when you need to be able to assign extra behaviors to objects at runtime without breaking the code that uses these objects.
- Use the pattern when it's awkward or not possible to extend an object's behavior using inheritance.

Pros:

- SRP
- Extending an objects behavior without making new sub classes
- Adding or removing responsibilities at runtime
- Combining several behaviors by wrapping an object into multiple decorators.

Cons:

- It's hard to remove a specific wrapper from the wrappers stack.
- It's hard to implement a decorator in such a way that its behavior doesn't depend on the order in the decorators stack.
- The initial configuration code of layers might look pretty ugly.

Decorator lets you change the skin of an object, while Strategy lets you change the guts.

4. Facade

Facade is a structural design pattern that provides a simplified interface to a library, a framework, or any other complex set of classes. A facade is a class that provides a simple interface to a complex subsystem which contains lots of moving parts.

Use Case:

- when you need to have a limited but straightforward interface to a complex subsystem.
- when you want to structure a subsystem into layers.

Pros:

- You can isolate your code from the complexity of a subsystem.

Cons:

- A facade can become a god object coupled to all classes of an app.

5. Proxy

Proxy is a structural design pattern that lets you provide a substitute or placeholder for another object. A proxy controls access to the original object, allowing you to perform something either before or after the request gets through to the original object. The Proxy pattern suggests that you create a new proxy class with the same interface as an original service object.

Use Case:

- Lazy initialization (**virtual proxy**) for heavyweight service objects to prevent wasting system resources
- Access control (**protection proxy**). This is when you want only specific clients to be able to use the service object
- Local execution of a remote service (**remote proxy**). This is when the service object is located on a remote server.
- Logging requests (**logging proxy**). This is when you want to keep a history of requests to the service object.
- Caching request results (**caching proxy**). This is when you need to cache results of client requests and manage the life cycle of this cache, especially if results are quite large.
- **Smart reference**. This is when you need to be able to dismiss a heavyweight object once there are no clients that use it.

Pros:

- OCP
- Controlling the service object without clients knowing
- Managing the lifecycle of the service object when clients don't care about it
- The proxy works even if the service object isn't ready or is not available.

Cons:

- Code may become complicated
- The response from the service might get delayed.

Behavioral Pattern

1. Chain of Responsibility

Chain of Responsibility is a behavioral design pattern that lets you pass requests along a chain of handlers. Upon receiving a request, **each handler decides either to process the request or to pass it to the next handler** in the chain.

Each linked handler has a field for storing a reference to the next handler in the chain. It's crucial that all handler classes implement the same interface.

Use Case:

- when your program is expected to process different kinds of requests in various ways, but the exact types of requests and their sequences are unknown beforehand.
- when it's essential to execute several handlers in a particular order.
- when the set of handlers and their order are supposed to change at runtime.

Pros:

- Controlling the order of request handling
- SRP
- OCP

Cons:

- Some requests may end up unhandled.

2. Observer

Observer is a behavioral design pattern that lets you define a subscription mechanism to notify multiple objects about any events that happen to the object they're observing.

Use Case:

- Use the Observer pattern when changes to the state of one object may require changing other objects, and the actual set of objects is unknown beforehand or changes dynamically.
- Use the pattern when some objects in your app must observe others, but only for a limited time or in specific cases.

Pros:

- OCP
- You can establish relations between objects at runtime.

Cons:

- Subscribers are notified in random order.

3. Strategy

Strategy is a behavioral design pattern that lets you define a family of algorithms, put each of them into a separate class, and make their objects interchangeable. The Strategy pattern suggests that you take a class that **does something specific in a lot of different ways** and extract all of these algorithms into separate classes called strategies.

Use Case:

- when you want to use **different variants of an algorithm** within an object and be able to **switch from one algorithm to another during runtime**.
- when you have a lot of similar classes that only differ in the way they execute some behavior.
- to isolate the business logic of a class from the implementation details of algorithms that may not be as important in the context of the logic
- when your class has a massive conditional statement that **switches between** different variants of the same algorithm. (Switch Smell)

Pros:

- Swapping algorithms at runtime
- Isolating the implementation details of an algorithm from the code
- Replacing inheritance with composition
- OCP

Cons:

- If you only have a couple of algorithms and they rarely change, there's no real reason to overcomplicate the program with new classes and interfaces that come along with the pattern.
- Clients must be aware of the differences between strategies to be able to select a proper one.
- A lot of modern programming languages have functional type support that lets you implement different versions of an algorithm inside a set of anonymous functions.

Template Method is based on inheritance: it lets you alter parts of an algorithm by extending those parts in subclasses. **Strategy is based on composition**: you can alter parts of the object's behavior by supplying it with different strategies that correspond to that behavior. Template Method works at the class level, so it's static. Strategy works on the object level, letting you switch behaviors at runtime.

4. Template Method

Template Method is a behavioral design pattern that defines the skeleton of an algorithm in the superclass but lets subclasses override specific steps of the algorithm without changing its structure. The Template Method pattern suggests that you **break down an algorithm into a series of steps, turn these steps into methods, and put a series of calls to these methods inside a single template method**.

There's another type of step, called hooks. A hook is an optional step with an empty body. A template method would work even if a hook isn't overridden. Usually, hooks are placed before and after crucial steps of algorithms, providing subclasses with additional extension points for an algorithm.

Use Case:

- when you want to let clients extend only particular steps of an algorithm, but not the whole algorithm or its structure.
- when you have several classes that contain almost identical algorithms with some minor differences. As a result, you might need to modify all classes when the algorithm changes.

Pros:

- You can let clients override only certain parts of a large algorithm, making them less affected by changes that happen to other parts of the algorithm.
- You can pull the duplicate code into a superclass.

Cons:

- Some clients may be limited by the provided skeleton of an algorithm.
- You might violate the Liskov Substitution Principle by suppressing a default step implementation via a subclass.
- Template methods tend to be harder to maintain the more steps they have.